# 임베디드 시스템

## Embedded Systems

Camera

FND

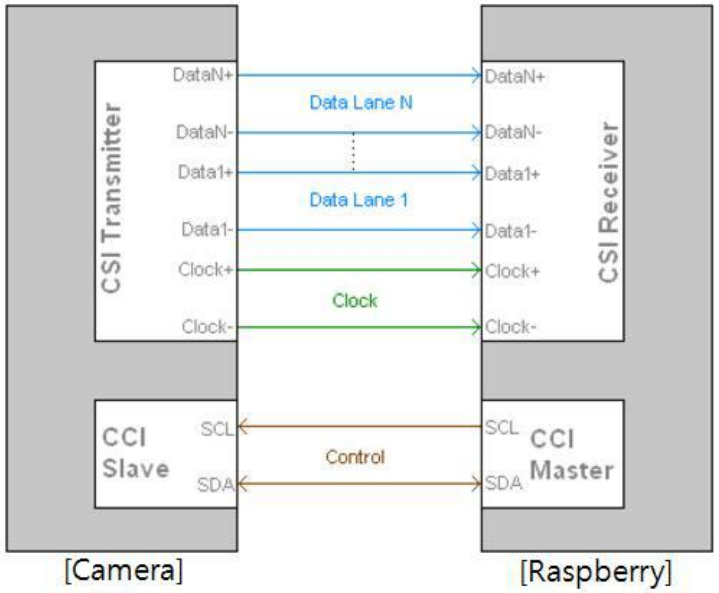
IR

강원대학교 컴퓨터공학과 권용훈

# Camera(1/5)

## Pi Camera 모듈

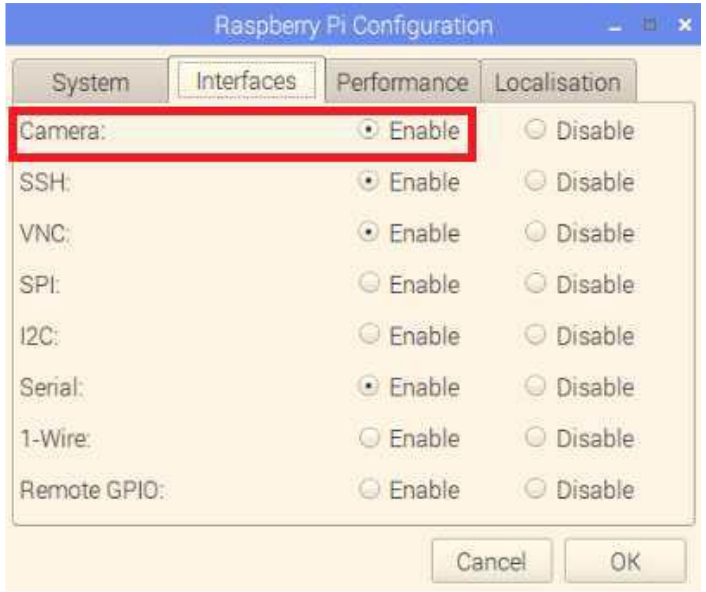
- 카메라 모듈은 리본 케이블을 사용해 연결
- I2C 방식을 통해서 데이터 송수신



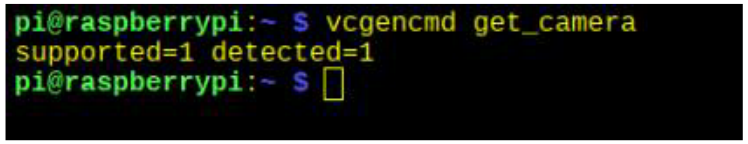

카메라 모듈의 통신 구조도

# Camera(2/5)

## Pi Camera 시스템 설정




시스템의 설정 과정




카메라 모듈 인식 명령어

# Camera(3/5)

## 명령어 – raspistill : 사진 촬영

```
Image parameter commands

-?, --help       : This help information
-w, --width      : Set image width <size>
-h, --height     : Set image height <size>
-q, --quality    : Set jpeg quality <0 to 100>
-r, --raw        : Add raw bayer data to jpeg metadata
-o, --output     : Output filename <filename> (to write to stdout, use '-o -').
-l, --latest     : Link latest complete image to filename <filename>
-v, --verbose    : Output verbose information during run
-t, --timeout    : Time (in ms) before takes picture and shuts down (if not specified, set to 5s)
-th, --thumb     : Set thumbnail parameters (x:y:quality) or none
-d, --demo       : Run a demo mode (cycle through range of camera options, no capture)
-e, --encoding   : Encoding to use for output file (jpg, bmp, gif, png)

Preview parameter commands

-p, --preview    : Preview window settings <'x,y,w,h'>
-f, --fullscreen         : Fullscreen preview mode
-op, --opacity   : Preview window opacity (0-255)
-n, --nopreview  : Do not display a preview window
```

pi@raspberrypi: ~/Pictures

```
pi@raspberrypi:~ $ raspistill -t 2000 -o ./Pictures/image01.jpg
pi@raspberrypi:~ $ cd ./Pictures/
pi@raspberrypi:~/Pictures $ ls
image01.jpg
```

명령어 예시

# Camera(4/5)

## 명령어 – raspvid : 영상 촬영

```
Image parameter commands

-?, --help      : This help information
-w, --width     : Set image width <size>. Default 1920
-h, --height    : Set image height <size>. Default 1080
-b, --bitrate   : Set bitrate. Use bits per second (e.g. 10MBits/s would be -b 10000000)
-o, --output    : Output filename <filename> (to write to stdout, use '-o -')
-v, --verbose   : Output verbose information during run
-t, --timeout   : Time (in ms) to capture for. If not specified, set to 5s. Zero to disabl
-d, --demo      : Run a demo mode (cycle through range of camera options, no capture)
-fps, --framerate       : Specify the frames per second to record
```

```
pi@raspberrypi: ~/Pictures
pi@raspberrypi:~/Pictures $ raspivid -o video.h264 -fps 30 -t 10000
pi@raspberrypi:~/Pictures $ ls
image01.jpg  video.h264
pi@raspberrypi:~/Pictures $
```

명령어 예시

# Camera(5/5)

[ EX_CameraSwitch.c ]

```c
//- 라이브러리 ----------------------------------------------------------------
#include <wiringPi.h>
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

//- 푸쉬 스위치 핀 선언 -------------------------------------------------------
#define CAPTURE_PUSH_RED_PIN    7                    //- Red Push Switch

//- Entry Function------------------------------------------------------------
int main(void)
{
        int oldPushedSW = HIGH, pushedSW = LOW;
        int idx = 1;

        //- wiringPi 초기화
        if( wiringPiSetup() == -1)
        {
                printf("Setup Fail ~!!");
                return -1;
        }

        //- Push Switch 동작 모드 설정
        pinMode(CAPTURE_PUSH_RED_PIN, INPUT);

        while(1)
        {
            pushedSW = digitalRead(CAPTURE_PUSH_RED_PIN);

            if((oldPushedSW != pushedSW) && (pushedSW == LOW))
            {
              char cmd[] = "raspistill -w 320 -h 240 -o /home/pi/Pictures/test";
              char file[3]= { };

```

[ EX_CameraSwitch.c ]

```c
              sprintf(file, "%d", idx++);
              strcat(cmd, file);
              strcat(cmd, ".jpg");
              printf("%s ₩n", cmd);
              system(cmd);
              system("ls /home/pi/Pictures/");

              printf("Take a picture ₩n");
            }
            oldPushedSW = pushedSW;
        }
        return 0;
}
```

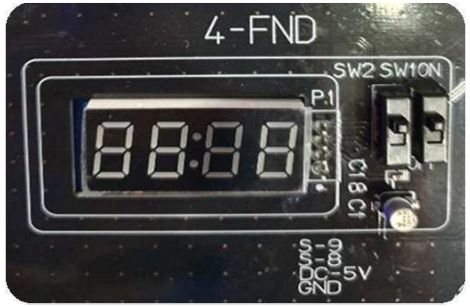
```
pi@raspberrypi:~/Example/CAMERA $ gcc -lwiringPi EX_CameraSwitch.c -o CAM
pi@raspberrypi:~/Example/CAMERA $ ls
CAM  EX_CameraSwitch.c
```

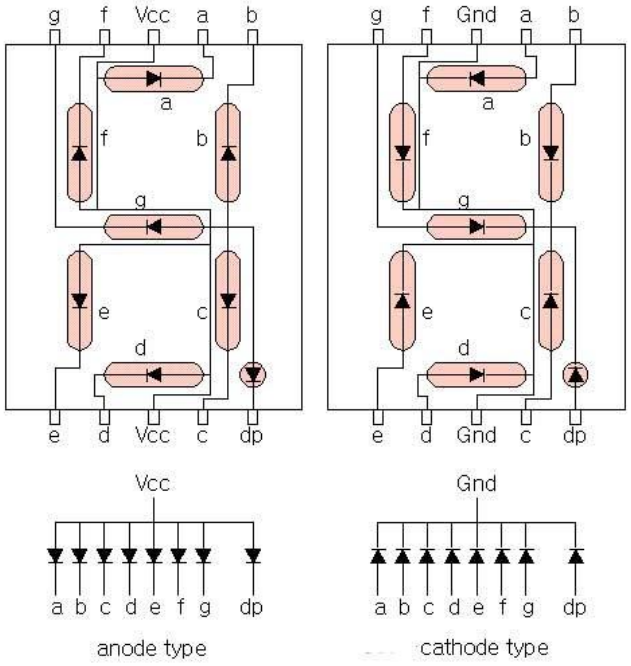
```
pi@raspberrypi:~/Example/CAMERA $ ./CAM
raspistill -w 320 -h 240 -o /home/pi/Pictures/test1.jpg
Take a picture
```

# 4-FND(1/6)

## FND(Flexible Numeric Display)

- 7-segment: 7개의 LED 소자를 8자 모양으로 배치하여 0~9까지 숫자를 표현하는 장치
- VCC와 GND에 연결된 방식에 따라 캐소드(Cathode)와 애노드(Anode)로 구분

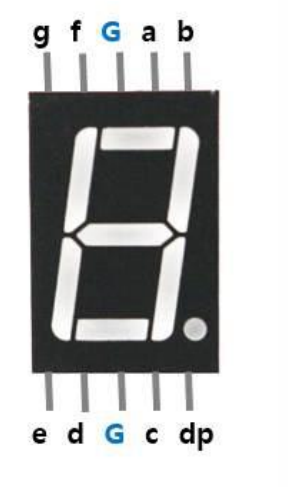


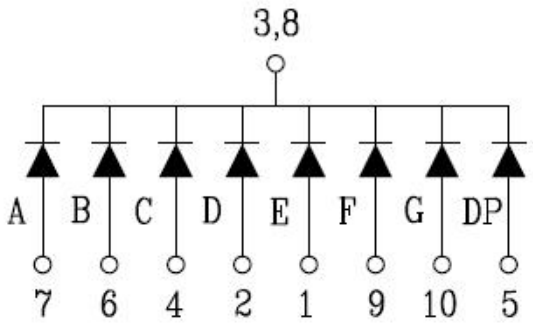



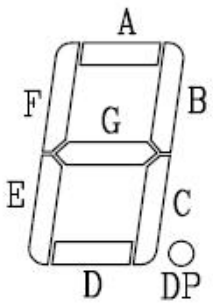
# 4-FND(2/6)

## 7-Segment의 원리

- 캐소드 타입
- 각 LED 소자의 공통 단자는 GND에 연결되어 있다.
- 각 LED를 켜기 위해선 HIGH, 끄기 위해선 LOW를 인가한다.



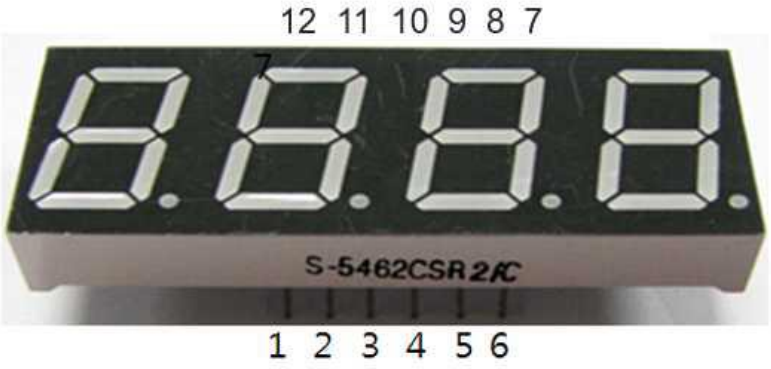

| 문자<br>숫자 | a<br>(PC5) | b<br>(PC4) | c<br>(PC2) | d<br>(PC1) | e<br>(PC0) | f<br>(PC6) | g<br>(PC7) | dp<br>(PC3) |
|---|---|---|---|---|---|---|---|---|
| 0 | 1 | 1 | 1 | 1 | 1 | 1 | 0 | 0 |
| 1 | 0 | 1 | 1 | 0 | 0 | 0 | 0 | 0 |
| 2 | 1 | 1 | 0 | 1 | 1 | 0 | 1 | 0 |
| 3 | 1 | 1 | 1 | 1 | 0 | 0 | 1 | 0 |
| 4 | 0 | 1 | 1 | 0 | 0 | 1 | 1 | 0 |
| 5 | 1 | 0 | 1 | 1 | 0 | 1 | 1 | 0 |
| 6 | 0 | 0 | 1 | 1 | 1 | 1 | 1 | 0 |
| 7 | 1 | 1 | 1 | 0 | 0 | 0 | 0 | 0 |
| 8 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 0 |
| 9 | 1 | 1 | 1 | 0 | 0 | 1 | 1 | 0 |
| a | 1 | 1 | 1 | 1 | 1 | 0 | 1 | 0 |
| b | 0 | 1 | 0 | 1 | 1 | 1 | 1 | 0 |
| c | 1 | 0 | 0 | 1 | 1 | 1 | 0 | 0 |
| d | 0 | 1 | 1 | 1 | 1 | 0 | 1 | 0 |
| e | 1 | 0 | 0 | 1 | 1 | 1 | 1 | 0 |
| f | 1 | 0 | 0 | 0 | 1 | 1 | 1 | 0 |

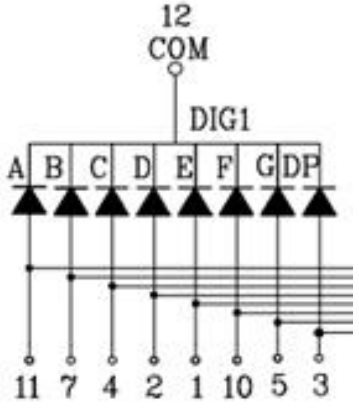
7-Segment 문자 & 숫자 표시 값

# 4-FND(3/6)

## 4-Digit 7-Segment

- 7개의 LED 핀과 더불어 각 자리를 제어하는 핀이 추가되었다.



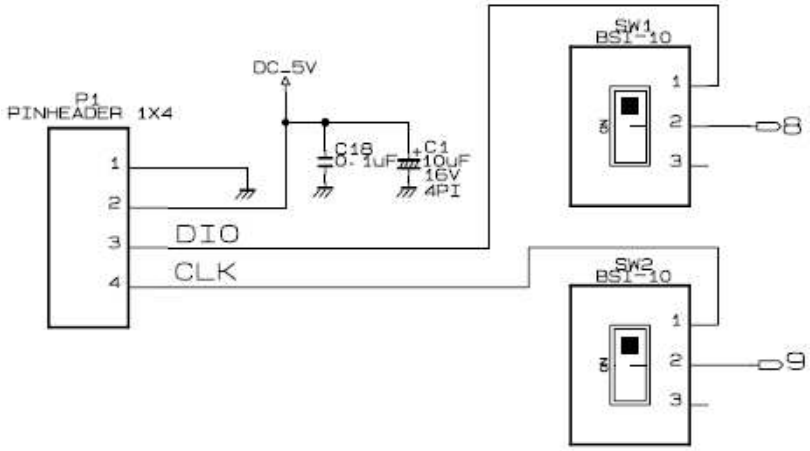




| 모듈 핀 | Segment |
|---|---|
| 1 | E segment |
| 2 | D segment |
| 3 | DP segment |
| 4 | C segment |
| 5 | G segment |
| 6 | **COMMENT** |
| 7 | B segment |
| 8 | **COMMENT** |
| 9 | **COMMENT** |
| 10 | F segment |
| 11 | A segment |
| 12 | **COMMENT** |

4-digit 7 seg의 핀 맵

# 4-FND(4/6)

## 4-FND의 연결회로




```
+-----+-----+---------+------+---+---Pi 3B+--+---+------+---------+-----+-----+
| BCM | wPi |   Name  | Mode | V | Physical | V | Mode | Name    | wPi | BCM |
+-----+-----+---------+------+---+----++----+---+------+---------+-----+-----+
|     |     |    3.3v |      |   |  1 || 2  |   |      | 5v      |     |     |
|   2 |   8 |   SDA.1 | ALT0 | 1 |  3 || 4  |   |      | 5v      |     |     |
|   3 |   9 |   SCL.1 | ALT0 | 1 |  5 || 6  |   |      | 0v      |     |     |
|   4 |   7 | GPIO. 7 |   IN | 1 |  7 || 8  | 0 | IN   | TxD     | 15  | 14  |
|     |     |      0v |      |   |  9 || 10 | 1 | IN   | RxD     | 16  | 15  |
|  17 |   0 | GPIO. 0 |   IN | 1 | 11 || 12 | 1 | IN   | GPIO. 1 | 1   | 18  |
|  27 |   2 | GPIO. 2 |   IN | 0 | 13 || 14 |   |      | 0v      |     |     |
```

# 4-FND(5/6)

## TM1637 라이브러리

| void TM1637_init(uint8_t Clk, uint8_t Data) | | |
|---|---|---|
| 위 치 | TM1637.h | |
| 기 능 | pwm 전용 핀에 pwm파형 출력 | |
| 매개변수 | uint8_t Clk | 클럭 핀 지정 |
| | uint8_t Data | 데이터 송수신 핀 지정 |
| 리턴 값 | void | 없음 |

| void TM1637_writeByte(int8_t wr_data) | | |
|---|---|---|
| 위 치 | TM1637.h | |
| 기 능 | software pwm 핀에 PWM 출력 | |
| 매개변수 | uint8_t Data | 8비트 데이터를 TM1637에 보내기 |
| 리턴 값 | void | 없음 |

| void TM1637_start(void) | | |
|---|---|---|
| 위 치 | TM1637.h | |
| 기 능 | 일반 핀에 pwm파형 출력을 정지 | |
| 매개변수 | void | 없음 |
| 리턴 값 | void | 없음 |

| void TM1637_stop(void) | | |
|---|---|---|
| 위 치 | TM1637.h | |
| 기 능 | 일반 핀에 pwm파형 출력을 정지 | |
| 매개변수 | void | 없음 |
| 리턴 값 | void | 없음 |

| void TM1637_set(uint8_t brightness,uint8_t SetData,uint8_t SetAddr) | | |
|---|---|---|
| 위 치 | TM1637.h | |
| 기 능 | 일반 핀에 pwm파형 출력을 정지 | |
| 매개변수 | uint8_t brightness | LED 밝기 설정<br>#define BRIGHT_DARKEST 0<br>#define BRIGHT_TYPICAL 2<br>#define BRIGHTEST 7 |
| | uint8_t SetData | 데이터 읽기 또는 쓰기 모드 설정<br>주소 지정 모드, 동작 모드 설정<br>0x40=>01000000 : 쓰기, 자동어드레싱, 일반 모드 |
| | uint8_t SetAddr | 디스플레이 레지스터 주소 설정 |
| 리턴 값 | void | 없음 |

# 4-FND(6/6)

[ EX_4_FND.c ]

```c
#include <stdio.h>
#include <wiringPi.h>
#include "TM1637.h"

#define CLK_PIN  9
#define DIO_PIN  8

int main(void){

  int8_t NumTab[] = {0,1,2,3,4,5,6,7,8,9,10,11,12,13,14,15};
  int8_t ListDisp[4];
  unsigned char i = 0,  count = 0;
  delay(150);

  //- wiringPi 초기화
  if( wiringPiSetup() == -1)
  {
        printf("Setup Fail ~!!");
        return -1;
  }
  //- 2개 핀 동작 모드 설정
  pinMode(CLK_PIN,INPUT);
  pinMode(DIO_PIN,INPUT);
  delay(200);

  TM1637_init(CLK_PIN,DIO_PIN);
  TM1637_set( BRIGHTEST, 0x40, 0xc0 );
```

[ EX_4_FND.c ]

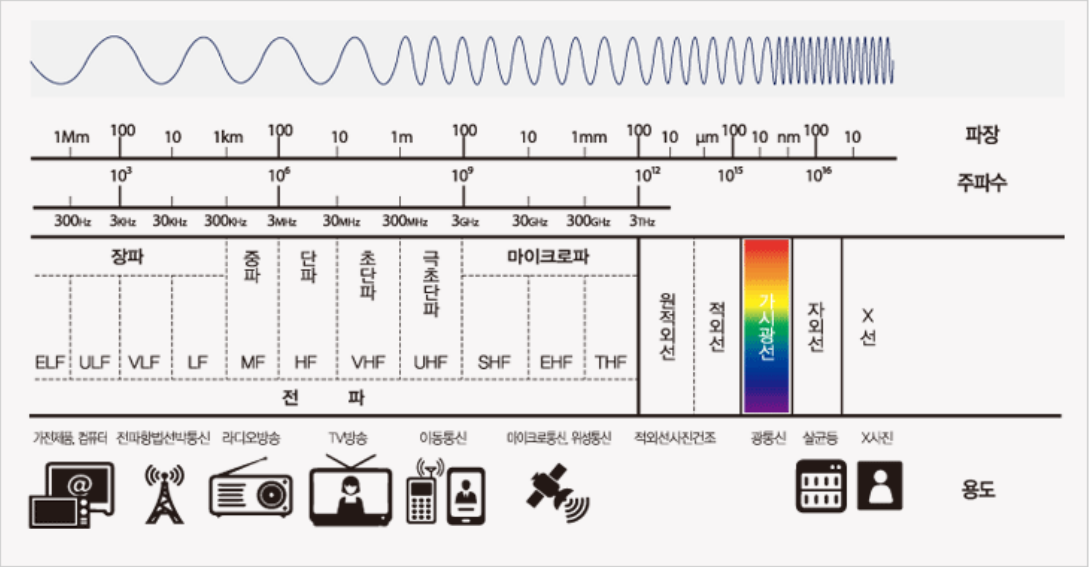
```c
        while(1)
        {
                unsigned char BitSelect = 0;
                i = count;
                count ++;

                if(count == sizeof(NumTab)) count = 0;

                for(BitSelect = 0;BitSelect < 4;BitSelect ++)
                {
                        ListDisp[BitSelect] = NumTab[i];
                        i++;

                        if(i == sizeof(NumTab)) i = 0;
                }

                TM1637_display(0,ListDisp[0]);
                TM1637_display(1,ListDisp[1]);
                TM1637_display(2,ListDisp[2]);
                TM1637_display(3,ListDisp[3]);
                delay(500);
        }
        return 0;
}
```

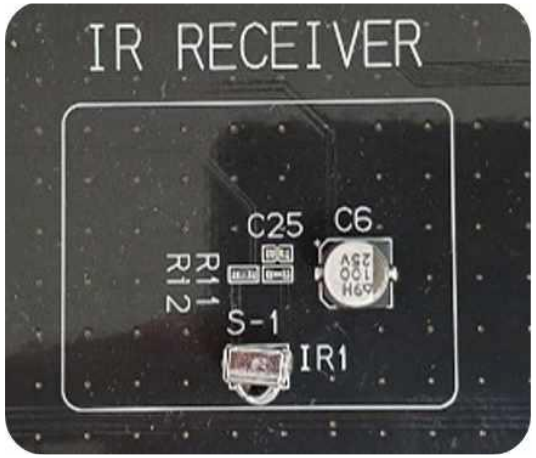
```
pi@raspberrypi:~/Example/GPIO $ gcc -lwiringPi EX_4_FND.c TM1637.c  -o FND-4
pi@raspberrypi:~/Example/GPIO $ ls
4FND   EX_4_FND.c    EX_PIR.c         EX_Ultrasonic.c  RGB_LED   ULT
BUZ    EX_Buzzer.c   EX_PushSwitch.c  FND-4            STEP
DCM    EX_DCMotor.c  EX_RGB_LED.c     PIR              TM1637.c
DHT    EX_DHT.c      EX_StepMotor.c   PS               TM1637.h
```

# IR(1/4)

## IR Receiver

- 적외선 리모컨 또는 적외선 송신기에서 보내는 적외선 신호를 받는 장치
- 자연광의 IR의 간섭을 피하기 위해 38KHz 주파수를 이용
- 통상적으로 NEC 포멧을 이용한다.

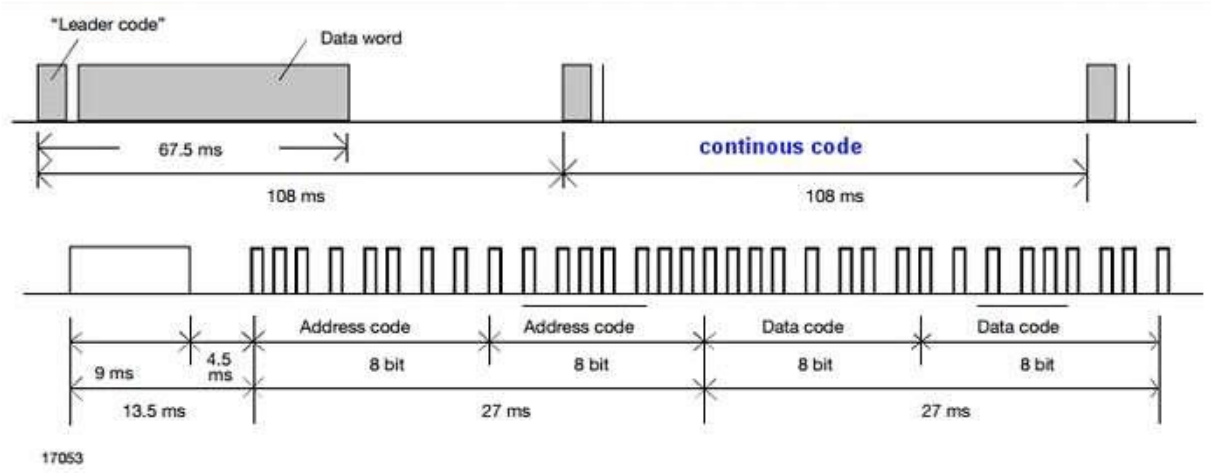


# IR(2/4)

## NEC

- Leader Code(9ms) + Space(4.5ms) + Data Word(54ms)
- 32bit = Address code(8bit) + Address Code1의 보수(8bit) + Data code(8bit) + Data code 1의 보수(8bit)



# IR(3/4)

## PIGPIO 설치

- 아래의 절차를 순차적으로 수행
- wget https://github.com/joan2937/pigpio/archive/master.zip
- unzip master.zip
- cd pigpio-master
- make
- sudo make install
- 출처: https://abyz.me.uk/rpi/pigpio/download.html

# IR(4/4)

**[ IR_receiver.c ]**

```
#include "irpigpio.h"
#include <stdio.h>
#define IR_PIN          18

int deviceSetup(){
 if (gpioInitialise()<0 || wiringPiSetup()<0){
        return -1 ;
    }
    /* IR pin as input */
    gpioSetMode(IR_PIN, PI_INPUT);
    /* 5ms max gap after last pulse */
    gpioSetWatchdog(IR_PIN, 5);
    /* monitor IR level changes */
    gpioSetAlertFunc(IR_PIN, alert);
}

int main(){
        long value;

        if(deviceSetup()<0)
                return 1;

        while(1){
                value = readIR();
                printf("IR value  %u₩n", value);
        }

        gpioTerminate();
}
```

**[ IR_LED.c ]**

```
#include "irpigpio.h"
#include <stdio.h>
#define IR_PIN          18
#define LED             27
#define BUTTON_0        951308750
#define BUTTON_1        271208262

int deviceSetup(){
 if (gpioInitialise()<0 || wiringPiSetup()<0){
        return -1 ;
    }
    /* IR pin as input */
    gpioSetMode(IR_PIN, PI_INPUT);
    /* 5ms max gap after last pulse */
    gpioSetWatchdog(IR_PIN, 5);
    /* monitor IR level changes */
    gpioSetAlertFunc(IR_PIN, alert);
}

int main(){
        long value;

        if(deviceSetup()<0)
                return 1;

        pinMode(LED, OUTPUT);

        while(1){
                value = readIR();
                switch(value){
                case BUTTON_0:printf("Button 0 pressed, LED On₩n");
                        digitalWrite(LED, HIGH);
                        break;
                case BUTTON_1:printf("Button 1 pressed, LED Off₩n");
                        digitalWrite(LED, LOW);
                        break;
                }
        }
        gpioTerminate();
}
```

# 고생하셨습니다

Thank you!